

Linux mm 新手学习路线图

从历史版本到当前主线（基于当前源码树 v7.0.0-rc7）

Codex 生成

2026-04-11

这份 PPT 的定位

目标

把一个对 Linux 内核 mm 子系统几乎零基础的人，带到能够：

- 看懂当前主线 mm/、Documentation/mm/ 的主要结构。
- 读旧资料时知道哪些概念还有效，哪些术语已经换代。
- 能沿着“分配、缺页、页缓存、回收、控制”五条主线跟代码。
- 具备继续深入某个方向并开始提交小补丁的能力。

范围

- 主线聚焦 MMU 系统；nommu 只做背景说明，不作为新手主线。
- 以当前源码树 7.0.0-rc7 为“现在”的参考点。
- 版本演进强调“理解结构变化的主线”，不是穷举每一个历史提交。

先把 mm 看成四层地图

第 1 层：物理内存

- node / zone / page frame / folio
- buddy allocator
- per-cpu pageset
- compaction / migration / hotplug

第 3 层：文件与缓存

- page cache / readahead
- writeback / dirty throttling
- shmem / tmpfs
- folio 化后的 file mm 路径

第 2 层：虚拟地址空间

- mm_struct
- vm_area_struct
- page tables
- page fault / COW / GUP / rmap

第 4 层：压力与控制

- reclaim / swap / OOM
- memcg / NUMA policy
- THP / hugetlb / multigen LRU
- DAMON / HMM / memory tiers

当前主线源码树里应该先记住的入口

主题	关键文档	关键代码
总体入口	Documentation/mm/index.rst Documentation/admin-guide/mm/ index.rst	mm/ include/linux/mm_types.h include/linux/mmzone.h
物理内存与分配	Documentation/mm/physical_ memory.rst Documentation/mm/page_ allocation.rst	mm/page_alloc.c mm/compaction.c mm/mm_init.c mm/memblock.c
地址空间与缺页	Documentation/mm/process_addrs. rst Documentation/mm/page_tables.rst	mm/mmap.c mm/vma.c mm/memory.c mm/rmap.c
页缓存与文件路径	Documentation/mm/page_cache.rst Documentation/mm/vmalloc.rst	mm/filemap.c mm/readahead.c mm/truncate.c mm/shmem.c

第一批要背下来的关键函数

分配 / 释放

- `alloc_pages()`
- `__alloc_pages()`
- `rmqueue()`
- `free_unref_page()`

地址空间 / 缺页

- `handle_mm_fault()`
- `do_anonymous_page()`
- `do_wp_page()`
- `finish_fault()`

回收 / 交换

- `shrink_node()`
- `shrink_folio_list()`
- `try_to_unmap()`
- `swap_read_folio()`
- `swap_writeout()`

现代特性

- `khugepaged`
- `lru_gen_*`
- `damon_*`
- `hmm_range_fault()`

读旧资料时最容易卡住的“术语断层”

旧资料里常见说法	当前主线更常见说法	你应该怎么理解
bootmem	memblock	很多旧书和老文章先讲 bootmem；今天读当前主线，应把“早期启动内存管理”优先映射到 memblock。
structpage 作为一切文件缓存对象	folio 在 file mm / page cache 路径中越来越主导	旧资料讲 page cache 往往全部用 page；现在很多 file mm 路径应优先按 folio 心智模型来读。
VMA 红黑树 / mm->mm_rb	mm_struct 中使用 Maple Tree	旧资料讲“查找 VMA 主要靠 rb-tree”；当前主线读 Documentation/mm/process_addrs.rst 时应转成 Maple Tree 视角。
active / inactive LRU 就是 reclaim 的全部故事	当前主线可能启用 multigen LRU，旧模型仍有帮助但不是唯一解释	旧 reclaim 文章仍值得读，但要知道现代回收路径有新观测接口和新策略。
高端内存 / highmem 是核心主题	在现代 64 位主线上，重点更多转向 folio、reclaim、memcg、heterogeneous memory	高端内存仍是历史知识点，但不应占用新手前期过多精力。

版本演进总览：先抓“理解主线”

时代	代表性变化	对新手理解的意义
Linux 0.x / 1.x	从 386 分页世界出发, swapper_pg_dir、基本 buddy、基本 demand paging	把 mm 先看成“页表 + 页框分配器 + 简单缺页”。
2.0 / 2.2 / 2.4	更大内存、更强 SMP 压力, zone / highmem / page cache 重要性上升	开始形成“物理内存布局 + 虚拟地址空间 + 缓存与回收”三件事一起看。
2.6 大时代	rmap、NUMA、memcg、KSM、compaction、migration、hotplug、slab 家族扩展	mm 从“分页实现”成长为“系统级内存平台”。
2.6.38 到 4.x	THP、work-ingset/refault、zswap、5-level page tables 等性能与可扩展性主题	开始必须把性能、回收策略、TLB、碎片整理一起理解。

阶段 1: Linux 0.x / 1.x 的 mm 该怎么看

- 这是“最小可理解模型”的时代：页表、页框、缺页、中断、换入换出。
- 当前文档 `Documentation/mm/page_tables.rst` 仍明确提到：早期 Linux 1.0 使用单个顶层页表 `swapper_pg_dir`，覆盖的是当时非常典型的 4MB 物理内存世界。
- 对新手最有价值的地方不是去背每个历史实现细节，而是意识到：
 - 起点非常 x86 / 386 导向。
 - 页表层级比今天浅得多。
 - 当时 mm 规模远小于今天，主问题也更单纯。
- 学历史版本的正确姿势：
 - 先学“最小模型”为何成立。
 - 再看今天为什么必须引入更多层：NUMA、memcg、THP、device memory、Maple Tree、MGLRU。

阶段 2: 2.0 / 2.2 / 2.4, mm 开始面对“真实规模”

- 更大的 RAM、更复杂的 SMP、更多文件缓存与 I/O 压力, 让 mm 不再只是“页表实现”。
- 这一阶段你应该重点抓住三条历史线索:
 - ① **zone / highmem 视角出现并重要化**: 物理内存不再被视作单一池子。
 - ② **page cache 重要性急剧上升**: 文件 I/O、mmap、缓存重用开始成为核心主题。
 - ③ **回收问题变成系统问题**: 什么时候回收、回收谁、代价是什么, 不再是边角问题。
- 如果你读到老资料大量讨论 `ZONE\_DMA`、`ZONE\_HIGHMEM`、`buffer/page cache` 分工, 不要惊讶; 这些都是从“更大机器和更复杂 I/O”逼出来的。
- 今天学这一段历史的意义是: 理解为什么现代 mm 永远在平衡“可分配性、可访问性、可回收性、可迁移性”。

阶段 3: 2.6 是 mm 真正“长成体系”的时代

- 如果说早期版本是“能工作”，那么 2.6 系列开始更像“要在各种规模机器上都能工作”。
- 新手应把这段时代记成六个关键词：
 - **rmap**: 反向映射让回收、迁移、拆页、失效同步变得更系统。
 - **NUMA**: node-local 分配、策略、拓扑意识成为常规主题。
 - **memcg**: 内存不再只按整机看，也要按 cgroup 控制和统计。
 - **compaction / migration**: 高阶分配和大页需要对抗碎片。
 - **KSM / hotplug / hugetlb**: mm 开始服务虚拟化、大规模部署、动态硬件。
 - **slab 家族演进**: 内核对象分配成为独立优化主题。
- 读这段历史的最佳态度：不要追求“一次看懂 2.6 全部”，而是认识到现代 mm 的大多数支柱都在这时成型。

阶段 4: 2.6.38 到 4.x, 性能模型开始主导设计

- **THP** 把“大页是否值得、如何整理碎片、何时折叠/拆分”推到前台。
- **workingset / refault** 让 reclaim 不再只是静态 LRU, 而是更关注访问历史和“错误驱逐”的代价。
- **zswap / frontswap** 一类技术说明 swap 也开始从“要不要用”变成“如何减轻代价”。
- **更深页表层级**说明虚拟地址空间和平台规模继续增长, 旧的简单页表想象已不足够。
- 对学习者最关键的变化:
 - TLB、页表层级、huge page、内存碎片、直接回收和后台回收开始彼此强耦合。
 - 你不能只会看“功能正确”, 还要会看“路径上的性能代价”。

阶段 5: 5.x / 6.x 到当前主线，现代化重构全面出现

- **HMM / ZONE_DEVICE**: mm 不再只管传统 DRAM，还要把 GPU / device memory 融入统一内存语义。
- **DAMON**: 访问模式观测从“只靠传统统计”转向“更细粒度、可编排的监测框架”。
- **folio** (5.16 前后开始成为关键词): file mm / page cache 路径中的基本对象表达方式发生明显变化。
- **Maple Tree** (6.1 前后成为地址空间主视角): VMA 管理不应再用老的红黑树心智模型硬套。
- **multigen LRU** (5.18 前后进入主线并持续成熟): 现代 reclaim 解释需要新的代际视角。
- 对新手的结论只有一句话: 旧资料仍然有价值，但你必须在脑中完成“旧术语 → 当前主线实现”的翻译。

当前这棵 7.0-rc7 树里，主导思想是什么

你现在读代码时应该优先带着的心智模型

- **对象层面**：file mm 优先想到 folio，不是旧式“什么都叫 page”。
- **地址空间层面**：VMA 已是 Maple Tree 语义世界，不是旧书里的 mm_rb 世界。
- **回收层面**：active/inactive LRU 仍是背景知识，但现代主线还要理解 multigen LRU。
- **控制层面**：memcg、NUMA、sysfs、debugfs、tracepoint、page owner 等可观测性非常重要。
- **硬件层面**：mm 需要同时面对 DRAM、huge pages、hotplug、device memory、tiered memory。

一句话总结

今天的 mm 已经不是“分页子系统”，而是“统一协调分配、映射、缓存、回收、隔离、监控与异构内存的基础平台”。

学习总策略：不要一上来就淹死在细节里

- ❶ 先建立总地图：node / zone / folio / VMA / page tables / reclaim。
- ❷ 再抓 5 条主路径：分配、缺页、页缓存、回收、控制。
- ❸ 每学一段都做实验，不做“纯阅读型学习”。
- ❹ 旧资料只用来解释“为什么会变成今天这样”，不要拿旧术语直接套当前代码。
- ❺ 始终把文档、代码、运行时观测三件事绑在一起。

建议周期

完整从零到“可以独立读主线路径并做小改动”，按兼职学习强度建议 **16 周**；如果全职投入，可压缩到 8 到 10 周，但不建议再短。

准备阶段：第 0 周先搭好学习环境

任务	要做什么	产出标准
源码导航	学会 rg、gitgrep、ctags/cscope/LSP 的最小用法	能在 1 分钟内定位 alloc_pages、handle_mm_fault、filemap_fault、shrink_node。
内核配置	准备一个带调试项的配置：CONFIG_DEBUG_VM、CONFIG_PAGE_OWNER、CONFIG_PAGE_TABLE_CHECK、CONFIG_MEMCG、CONFIG_LRU_GEN、CONFIG_TRANSPARENT_HUGEPAGE	你能解释每个选项是为了观测什么。
运行环境	最好准备 qemu/kvm 或可重启测试机，避免只在生产机观察	有一套能安全触发内存压力、交换、THP 的实验环境。
基础概念	先读 Documentation/admin-guide/mm/concepts.rst、Documentation/mm/page_tables.rst、Documentation/mm/physical_memory.rst	能够口头解释 PFN、TLB、zone、node、anon、page cache、reclaim。

第 1-2 周：补硬件与页表基础

维度	内容	最终标准
核心目标	把“虚拟地址 → 页表 → 物理页框 → TLB”的链条补齐	不再把页表、缺页、权限位、TLB 当成抽象口号。
必读文档	Documentation/admin-guide/mm/ concepts.rst Documentation/mm/page_tables.rst	知道五级页表是当前软件层次表达；知道哪些层可能 fold。
关键代码	include/linux/pgtable.h include/linux/mm_types.h arch/x86/mm/fault.c (若以 x86_64 为主)	能区分架构相关 fault 入口和通用 mm 路径。
实验	跟一次用户态缺页，记录“地址、VMA、PTE、folio/page”的变化	能画出匿名页首次访问和写时复制的基本时序图。

第 3-4 周：物理内存、zone、buddy、PCP

维度	内容	最终标准
核心目标	搞清 Linux 如何把物理内存表示为 node / zone / free area / order / PCP	能从 gfp 标志一路解释到“最终从哪个 zone 取页”。
必读文档	Documentation/mm/physical_memory.rst Documentation/mm/page_allocation.rst	理解 watermarks、zonelist、movable zone、大页分配失败为何常与碎片有关。
关键代码	mm/page_alloc.c include/linux/mmzone.h include/linux/gfp_types.h	能读主分配路径，不被所有 fastpath / slowpath 细节立刻淹没。
实验	看 /proc/zoneinfo、/proc/buddyinfo、/proc/pagetypeinfo，观察压力前后变化	能把“高阶分配失败”与“碎片、watermark、fallback”联系起来。

第 5-6 周: slab/slub、vmalloc、早期内存

维度	内容	最终标准
核心目标	区分“页分配器”和“内核对象分配器”； 理解早期启动期与运行期分配方式不同	不再把 <code>kmalloc</code> 、 <code>alloc_pages</code> 、 <code>vmalloc</code> 混为一谈。
必读文档	<code>Documentation/mm/slab.rst</code> <code>Documentation/admin-guide/mm/slab.rst</code> <code>Documentation/mm/vmalloc.rst</code>	知道对象分配、线性映射之外的虚拟映射、cache 热路径分别在解决什么问题。
关键代码	<code>mm/slub.c</code> <code>mm/slab_common.c</code> <code>mm/vmalloc.c</code> <code>mm/memblock.c</code> <code>mm/mm_init.c</code>	知道旧资料里的 <code>bootmem</code> 今天如何映射为 <code>memblock</code> 世界。
实验	读 <code>/proc/slabinfo</code> ，观察常见 cache； 触发不同尺寸 <code>kmalloc</code> ；区分 <code>vmalloc</code> 和线性映射地址	能解释为什么“连续虚拟地址”不等于“连续物理地址”。

第 7-8 周: mm_struct、VMA、page fault、COW

维度	内容	最终标准
核心目标	学会从进程地址空间视角读 mm: mm_struct、VMA、页表和 fault 是怎样连起来的	能独立讲清匿名缺页、文件缺页、写时复制的主路径。
必读文档	Documentation/mm/process_addrs.rst Documentation/mm/page_tables.rst	知道当前主线里 VMA 查询/组织要按 Maple Tree 心智模型理解。
关键代码	mm/mmap.c mm/vma.c mm/memory.c mm/mprotect.c mm/rmap.c	能在源码里顺着 fault 主路径走到 PTE 建立、COW、rmap。
实验	用 mmap()、fork()、写入触发 COW; 观察 /proc/<pid>/maps、smaps、pagemap	能把 VMA、页表项、物理页和进程行为对应起来。

第 9–10 周: page cache、readahead、writeback、folio

维度	内容	最终标准
核心目标	把“文件 I/O 为什么会进入 mm”这件事真正搞懂	知道 page cache 为何是 Linux 文件路径和内存路径之间的核心桥梁。
必读文档	Documentation/mm/page_cache.rst Documentation/mm/process_addrs.rst (file-backed VMA 部分)	理解当前文档已经明确把 folio 作为 page cache 的基本管理单位。
关键代码	mm/filemap.c mm/readahead.c mm/truncate.c mm/shmem.c	能区分 buffered I/O、mmap file fault、readahead、dirty/writeback。
实验	对比 read() 与 mmap() 读文件; 观察缓存命中前后、预读和回收行为	能解释 folio 为什么能让 file mm 路径表达更清晰。

第 11–12 周: reclaim、swap、OOM、workingset

维度	内容	最终标准
核心目标	理解“内存不够时到底发生了什么”，包括后台回收、直接回收、交换和 OOM	能把 kswapd、direct reclaim、swap in/out、OOM 串成完整故事。
必读文档	Documentation/mm/page_reclaim.rst Documentation/mm/swap.rst Documentation/mm/oom.rst Documentation/admin-guide/mm/multigen_lru.rst	知道旧 LRU 叙述与当前 multigen LRU 观测接口之间的关系。
关键代码	mm/vmscan.c mm/workingset.c mm/swap.c mm/swapfile.c mm/oom_kill.c mm/zswap.c	能在代码中找到“为什么被回收”“为什么被换出”“为什么最后触发 OOM”。
实验	人为制造内存压力，观察 /proc/vmstat、/sys/kernel/mm/lru_gen/、swap 活动和 OOM 日志	能分辨“抖动”“正常回收”“已经进 OOM 边缘”三种状态。

第 13–14 周: THP、compaction、migration、NUMA、memcg

维度	内容	最终标准
核心目标	进入“真正像现代 mm”的主题: 大页、碎片整理、跨节点、分组控制	能把性能、可分配性、拓扑、隔离这四件事放在一起思考。
必读文档	<code>Documentation/mm/transhuge.rst</code> <code>Documentation/admin-guide/mm/transhuge.rst</code> <code>Documentation/mm/page_migration.rst</code> <code>Documentation/mm/numa.rst</code> <code>Documentation/admin-guide/mm/numa_memory_policy.rst</code>	知道 THP、mTHP、compaction、migration、NUMA policy、memcg 各自解决什么问题。
关键代码	<code>mm/huge_memory.c</code> <code>mm/khugepaged.c</code> <code>mm/compaction.c</code> <code>mm/migrate.c</code> <code>mm/mempolicy.c</code> <code>mm/memcontrol.c</code>	能顺着 THP 折叠或 compaction 失败路径读出约束条件。

第 15–16 周：现代主题收尾，建立“当前主线感”

维度	内容	最终标准
核心目标	把 modern mm 的新话语体系真正装进脑子：folio、Maple Tree、MGLRU、DAMON、HMM、memory tiers	能看懂近几年的主线 mm 讨论，不再只会引用老书。
必读文档	Documentation/mm/process_addrs.rst Documentation/mm/multigen_lru.rst Documentation/mm/damon/design.rst Documentation/mm/hmm.rst	知道当前 mm 为什么越来越强调可观测性、异构内存、现代数据结构。
关键代码	mm/vma.c mm/damon/ mm/hmm.c mm/memory-tiers.c	能指出这些主题分别作用在“地址空间”“回收策略”“异构设备”“冷热分层”的哪一层。
最终项目	做一个 mini project：写学习笔记、画主路径图、加 tracepoint/bpftrace 观测脚本，或修一个小 warning / 注释 / 文档问题	达到“能开始参与 mm 社区讨论”的最低门槛。

每周固定节奏：不要只读不练

- ① 周一：读文档，补概念，写 1 页笔记。
- ② 周二：从 1 个关键函数出发跟主调用链，最多追两层分支。
- ③ 周三：做 1 个实验，必须有观测结果，不接受“脑补运行”。
- ④ 周四：回到代码解释实验现象，补齐锁、数据结构、状态转换。
- ⑤ 周五：输出 1 张时序图或 1 张数据结构图。
- ⑥ 周末：复盘，把“旧术语”和“当前实现”做一次映射。

最重要的纪律

每学完一个主题，必须能回答三个问题：

- 这个机制处理的核心对象是什么？
- 它的快路径和慢路径分别在哪里？
- 它和 reclaim / fault / folio / memcg / NUMA 中的哪条线有耦合？

必须做的 8 个实验

- ❶ 匿名页首次访问与 COW: 观察 `fork()` 前后写入行为。
- ❷ zone / buddy 观测: 在低压和高压下对比 `/proc/buddyinfo`。
- ❸ `kmalloc` vs `alloc_pages` vs `vmalloc`: 区分对象、页、虚拟映射。
- ❹ buffered I/O vs `mmap()`: 观察 page cache 与 file fault。
- ❺ direct reclaim vs `kswapd`: 制造不同级别压力, 观察 stall。
- ❻ THP / mTHP: 切换 `always/madvise/never`, 看性能和内存碎片表现。
- ❼ memcg 压力: 在 cgroup 中跑 workload, 看回收与 OOM 是否局部化。
- ❽ multigen LRU: 查看 `/sys/kernel/mm/lru_gen/` 与 debug 接口, 理解冷热代际分布。

观测与调试工具箱

工具 / 接口	用来观察什么	何时最有用
/proc/meminfo、 /proc/vmstat	整机内存概况、换页、回收、脏页、OOM 前兆	刚开始做任何内存实验时的第一眼。
/proc/zoneinfo、 /proc/buddyinfo、 /proc/ pagetypeinfo	zone 水位、碎片、高阶分配可能性	怀疑 compaction / 高阶分配失败时。
/proc/<pid> /maps、smaps、 pagemap	地址空间、VMA、RSS、匿名/文件页分 布	研究 fault、COW、mmap、page cache 时。
/sys/kernel/mm/ lru_gen/	multigen LRU 开关与统计	读现代 reclaim 时必看。
tracefs / perf / bpftrace	路径级时序与热点	确认到底慢在哪一段。
page_owner、page_ table_check	页来源、非法页表问题	查泄漏、页表问题、可疑分配来源。

读 old mm 资料时，如何做“翻译”而不是照抄

正确姿势

- 如果老资料在讲 bootmem：立刻在当前树中对照 memblock。
- 如果老资料在讲 page cache 全部是 structpage：去当前 Documentation/mm/page_cache.rst 看 folio 语义。
- 如果老资料在讲 VMA 红黑树：回到 Documentation/mm/process_addr.rst，按 Maple Tree 视角重建理解。
- 如果老资料只讲双链表 LRU：再补 Documentation/mm/multigen_lru.rst 和 Documentation/admin-guide/mm/multigen_lru.rst。
- 如果老资料完全不提 memcg / HMM / DAMON：把它当“时代背景缺失”，不是你理解错了。

错误姿势

- 把 2.6 时代的术语原样套到 7.0 主线代码上。
- 只读经典书，不读当前文档和当前代码。

什么时候算“掌握了 mm”

- ① 你能解释 node、zone、page、folio、VMA、PTE、anon、page cache、rmap、reclaim、memcg、THP、HMM、DAMON。
- ② 你能独立跟完 4 条主路径：
 - 一次页分配
 - 一次匿名缺页
 - 一次文件缺页
 - 一次回收导致的换出或 OOM
- ③ 你能把旧资料与当前主线差异说清楚，而不是只会引用旧图。
- ④ 你能设计并执行 1 个内存实验，并用代码解释结果。
- ⑤ 你敢改一个小 patch：文档、注释、trace、统计、warning 修复，任意一种。

更现实的判断

“掌握 mm”不是一次性完成，而是先能读主路径，再选一个方向深钻：page allocator、reclaim、file mm、THP、memcg、NUMA、device memory 都可以各自深入很多年。

学到最后，建议你选一条“主修线”

如果你偏系统基础

- page allocator
- reclaim / vmscan
- compaction / migration
- memcg / OOM

如果你偏文件与缓存

- filemap / page cache
- readahead / writeback
- folio 化改造
- shmem / tmpfs

如果你偏进程地址空间

- mmap / VMA
- fault / COW
- GUP / rmap
- Maple Tree / locking

如果你偏现代方向

- THP / mTHP
- multigen LRU
- DAMON
- HMM / device memory / tiers

推荐阅读顺序

- ① `Documentation/admin-guide/mm/concepts.rst`
- ② `Documentation/mm/page_tables.rst`
- ③ `Documentation/mm/physical_memory.rst`
- ④ `Documentation/mm/process_addrs.rst`
- ⑤ `Documentation/mm/page_cache.rst`
- ⑥ `Documentation/mm/transhuge.rst`
- ⑦ `Documentation/mm/multigen_lru.rst`
- ⑧ `Documentation/mm/damon/design.rst`
- ⑨ `Documentation/mm/hmm.rst`
- ⑩ 最后再回头读旧书或老文章，做历史映射

补充建议

经典旧资料依然有价值，尤其适合建立“为什么当年要这样设计”的感觉；但只要你开始读当前主

一句话收尾

真正的学习路径

先把 Linux mm 从“神秘黑箱”变成一张可导航的地图；再把历史版本变迁当成理解现代设计的背景；最后通过实验和代码阅读，把这张地图变成你自己的工程直觉。

如果你严格按这份 16 周路线执行，到结束时，你不一定“精通整个 mm”，但已经足够进入主线语境，并开始做真正有价值的深入学习。